Agentic AI Coworkers

Daily technical learning and personalized career readiness

Student project · San José State University
Prepared for the October 15, 2026 agentic AI assignment
Prototype date: November 1, 2026

This report documents two executable Agentic AI Coworkers. The Python runner is the working prototype: it retrieves and ranks technical sources, analyzes opportunity channels, generates exercises, evaluates learner answers, records state transitions, and persists shared memory. The HTML dashboard is a visual demonstration layer, not the agent implementation.

Executive Summary

The project implements a two-agent learning and career loop for an SJSU student preparing for AI and machine-learning engineering opportunities. DailyTechUpgradeAgent produces a 15–20 minute brief from authoritative technical sources and connects each recommendation to the learner's active gaps. nextLevelPersonalCoachForCareerLevel analyzes a target role, identifies opportunity channels, asks technical questions, evaluates answers, generates a coding exercise and interview scenario, and stores the next preparation actions.

Agent	Primary outcome	Executable command
DailyTechUpgradeAgent	Daily Top 3 brief with source metadata, summary, relevance, and next action	python3 agentic_workers.py daily
nextLevelPersonalCoachForCareerLevel	Career readiness assessment with feedback, opportunities, exercise, and plan	python3 agentic_workers.py coach --interactive

1 Agent Goals

DailyTechUpgradeAgent

Goal: become a better technical learner every day through a focused, trustworthy reading habit.

- Discover three relevant technical articles or blog posts from authoritative sources.
- Summarize the technical idea and state one practical takeaway.
- Personalize ranking using interests, active knowledge gaps, target role, and prior learning memory.
- Keep the session within 15–20 minutes and end with an applied next action.

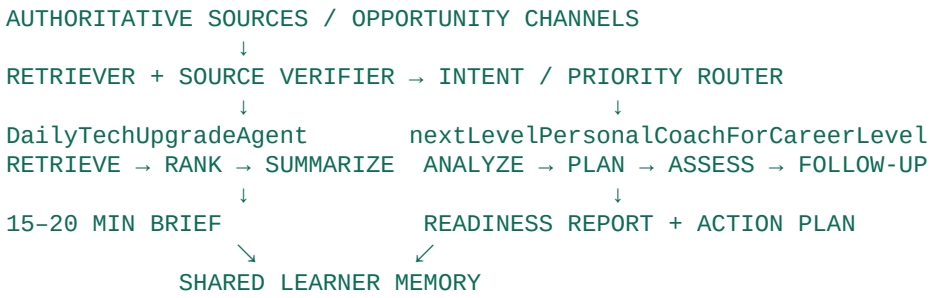
nextLevelPersonalCoachForCareerLevel

Goal: prepare the learner for the next internship, job, research, or entrepreneurial opportunity through evidence-based practice.

- Analyze a target role and identify internship, job, research, SJSU career, and startup opportunity channels.
- Generate technical questions, a coding exercise, and an interview scenario.
- Evaluate answers with a visible rubric, identify strengths and gaps, and ask a follow-up question.
- Persist progress, active gaps, completed sessions, and next actions in shared memory.

2 Architecture and Agent Workflow

Both workers use the same inspectable loop: observe learner memory, call tools, transition through explicit states, reflect against a constraint or rubric, and save the result.



Component	Role in the system	Implementation evidence
Source retriever	Collect live RSS items and verified fallback sources	AWS, LangChain, and Google Cloud RSS attempts; official fallback records
Opportunity analyzer	Map target role to concrete opportunity paths	LinkedIn search, SJSU Career Center, NSF REU, YC Co-Founder Matching
Personalization scorer	Rank content against interests and active gaps	Tag overlap score stored in each recommendation
Assessment engine	Evaluate technical answers and follow-ups	States, guardrails, recovery, persistence rubric
Model adapter	Generate richer summaries and feedback when configured	OpenAI Responses API via OPENAI_API_KEY; offline fallback otherwise
Memory store	Persist learner profile, events, briefs, sessions, and gaps	.agent_memory.json

3 Working Prototype

The prototype is dependency-free and runs from a terminal. It can use the OpenAI Responses API when an API key is configured, but it does not require a key for demonstration or grading.

Run both agents

```
cd "agentic coworkers"
python3 agentic_workers.py demo
```

Run the interactive coach

```
python3 agentic_workers.py coach --interactive

The coach asks a system-design question, evaluates the answer, asks a
safety/observability follow-up, and saves the session.
```

Optional model-backed mode

To use model-generated summaries and coaching feedback, set an API key in the terminal session. The key is read from the environment and is never written to project files.

```
export OPENAI_API_KEY="your-key"
python3 agentic_workers.py demo
```

4 Sample Results

Daily technical brief

Article	Source and date	Technical insight	Career connection
How we detect frontier model misalignment	OpenAI · Oct 29, 2026	Monitor agent reasoning, actions, and environmental effects to detect unauthorized behavior.	Builds safety and observability judgment for agent-system interviews.
Gemini at Work 2026: Introducing the Gemini agent	Google Cloud · Oct 8, 2026	Production agents require memory, identity, authorization, sandboxing, and observability.	Connects architecture decisions to secure deployment.
Advancing computer use with Ironclad	OpenAI · Oct 6, 2026	Professional agents need business rules, multi-step execution, and outcome verification.	Improves ability to explain reliable tool-using systems.

The fallback records link to the official source pages. When live RSS is reachable, the agent ranks current feed items first and preserves the same output schema.

Interactive career coaching session

Prompt: An agent may search the web, but a human must approve external side effects. Describe the state transitions, guardrails, and recovery path.

Example learner answer: I would use Ready, ToolProposed, AwaitingApproval, Executing, Succeeded, and Failed states. Permission checks happen before side effects. I would persist the trace and retry transient failures.

Rubric dimension	Result	Evidence
States	Pass	Ready, ToolProposed, AwaitingApproval, Executing, Succeeded, Failed
Guardrails	Pass	Permission and human approval before side effects
Recovery	Pass	Retry transient failures and represent Failed state
Persistence	Pass	Persist trace and approval decision

Generated coding exercise: implement an approval-aware agent trace function that returns READY, AWAITING_APPROVAL, EXECUTING, SUCCEEDED, or FAILED, rejects side effects without approval, and records a trace event.

Generated interview scenario: explain how the agent would be observed, secured, and evaluated in production.

5 Personalization and Memory

Memory field	Current value	How it changes behavior
Interests	AI agents, evaluation, Python APIs	Ranks articles with matching tags
Target role	AI/ML Engineer Intern	Shapes opportunity analysis and coaching rubric
Active gaps	Observability, production system design	Selects takeaways, coding exercise, and preparation plan
Milestone	Prepare a portfolio-ready agent system	Converts reading into an applied next action

Every run saves timestamps, state transitions, recommendations, assessment results, opportunities, exercises, and next actions. This makes progress inspectable rather than inferred from chat length.

6 Evaluation

Dimension	Metric	Target
Recommendation accuracy	Verified title, date, source, URL, and faithful summary	At least 90% citation correctness
Relevance	Learner rating of each article	Average of 4/5 or higher
Learning transfer	Pre/post answer on a novel technical prompt	Improvement in rubric score and application quality
Coach calibration	Agreement with a human rubric across 10 answers	At least 80% agreement
Career progress	Skill evidence, artifacts, and interview performance	One applied artifact per week
Workflow reliability	Completed runs and saved trace events	No silent failure; fallback is labeled

Test procedure: run the daily brief for ten days; rate freshness, relevance, and actionability; submit ten technical answers; compare agent feedback with a human rubric; then complete an unseen system-design prompt and compare it with the baseline.

7 Limitations and Responsible Use

- RSS availability varies by source; the agent labels its fallback behavior and preserves official source URLs.
- Readiness scores are coaching signals, not hiring predictions.
- The learner should inspect and correct stored memory and avoid entering sensitive personal information.
- External actions remain outside the prototype; a production deployment should require explicit approval, authentication, rate limits, and tool-level audit logs.

8 Submission Checklist

Required deliverable	Where it is provided	Status
Agent goals	Sections 1 and 2	Complete
Architecture diagram	Section 2	Complete
Working prototype	agentic_workers.py and commands in Section 3	Complete
Sample results	Section 4 and demo output	Complete
Evaluation	Section 6	Complete
Supporting dashboard	index.html	Complete